# TASK 1 – Understand the Problem and Identify the Data

## Problem analysis:

Grama Jyothi Rural Electricity Co-operative bills households by comparing the previous and current meter readings. Consumption is the number of units used during the billing period. The bill contains a fixed charge and an energy charge calculated using slabs: the first 100 units are free, units 101–200 cost ₹2.25 per unit, and units above 200 cost ₹4.50 per unit. The fixed charge is ₹50 in the supplied code example.

The program must avoid negative bills, calculate money exactly to the paisa, handle a 16-bit meter rollover, and keep tariff values in one place so a tariff revision cannot be applied inconsistently

| Data item | Purpose and justification | Suggested representation |
|---|---|---|
| Household number | Identifies the customer and the bill; must not be confused with consumption. | int or a suitably sized integer; string if it contains letters/leading zeroes |
| Category | Stores household/customer tariff category if different tariffs are later introduced. | enum or small integer |
| Previous reading | Meter register value at the start of the period; in the stated example it is 0–65535. | unsigned short / uint16_t where supported |

| | | |
|---|---|---|
| Current reading | Meter register value at the end of the period; same register range. | unsigned short / uint16_t |
| Units consumed | Difference between readings, including rollover logic. | unsigned int or uint32_t |
| Energy charge | Sum of the applicable slab charges. | long long integer paise |
| Fixed charge | Fixed amount added to each bill; constant under a given tariff. | long or long long integer paise |
| Total bill | Fixed charge plus energy charge; must be non-negative and exact to the paisa. | long long integer paise |
| Tariff rates and slab limits | Shared rules that should not be repeated throughout the program. | named macros or const values; configuration file in revised design |
| Number of bills / total collection | May grow from 4,000 to 800,000 households. | size_t for counts; long long for total paise |

**Inputs, outputs, rules and assumptions:**

- Inputs: household number, category, previous reading and current reading. Input validation must reject missing, malformed or out-of-range values.
- Outputs: units consumed, energy charge, fixed charge and final bill in rupees and paise.
- Rules: total bill = fixed charge + energy charge; all charges are non-negative; the first 100 units have zero energy charge; slab boundaries are applied exactly once; a rollover is handled according to the meter register's modulus.

- Assumption: the basic example uses a 16-bit register with values 0–65535 and one rollover between readings. Multiple rollovers cannot be inferred from only two register readings.
- Assumption: the sample tariff and fixed charge are those shown in the assignment code. Real-world tax, subsidies, late fees and category-specific rules are outside the supplied brief.

**Text strings versus integers: time and space:**

If a reading is stored as text such as "01530", the program must validate the characters and convert the digit sequence to a numeric value before calculating consumption. Conversion and comparison each take O(d) time for d digits. A fixed-width integer comparison/subtraction is O(1).

| Representation | Time per reading | Space per reading | Comments |
|---|---|---|---|
| Integer meter value | O(1) arithmetic | Fixed size, commonly 2 bytes for unsigned short | Best for calculations; leading zeroes are not retained. |
| Text such as "01530" | O(d) to validate/convert; O(d) to compare as text if numeric meaning is needed | At least d characters plus terminator for a C string | Preserves formatting, but requires conversion and validation. |

For the assignment's small fixed register, integer storage is simpler and faster. Text input may still be read as a line and then safely converted, but the value used for arithmetic should be numeric.

# TASK 2 – Choose the Data Types:

## Recommended types and portability:

| Item | Recommended type | Reason / range consideration |
|---|---|---|
| Meter reading | unsigned short or uint16_t | The brief specifies a 16-bit register. Confirm USHRT_MAX; uint16_t exists only where an exact 16-bit type is available. |
| Units consumed | unsigned int / uint32_t | Non-negative; wide enough for the difference between two 16-bit readings. |
| Household count / loop index | size_t or unsigned long long as appropriate | size_t is natural for array sizes; district-wide counts must not rely on a 16-bit int. |
| Single bill in paise | long long | Integer arithmetic avoids binary floating-point rounding. |
| Total collection in paise | long long / int64_t | A district total may exceed a 32-bit signed long. |
| Tariff rate in paise per unit | int or long | Small integer values, with a type chosen consistently for arithmetic. |
| Text category | enum or char array | enum is suitable for a fixed set of named categories; string if values come from external data. |

C does not guarantee that int, long or long long have the same size on every platform. Use sizeof to measure the local implementation and limits.h / float.h to inspect ranges. A common 64-bit Windows compiler uses 4-byte int and 4-byte long, while long long is 8 bytes; many 64-bit Unix-like systems use an 8-byte long. These are common examples, not universal guarantees.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void) {
    printf("sizeof(short) = %zu, USHRT_MAX = %u\n",
           sizeof(short), (unsigned)USHRT_MAX);
    printf("sizeof(int) = %zu, INT_MAX = %d\n",
           sizeof(int), INT_MAX);
    printf("sizeof(long) = %zu, LONG_MAX = %ld\n",
           sizeof(long), LONG_MAX);
    printf("sizeof(long long) = %zu, LLONG_MAX = %lld\n",
           sizeof(long long), LLONG_MAX);
    printf("FLT_DIG = %d, DBL_DIG = %d\n", FLT_DIG,
DBL_DIG);
    return 0;
}
```

**Why money is stored as integer paise:**

A rupee has 100 paise. A bill such as ₹162.50 is represented as 16,250 paise. Integer addition and multiplication are exact as long as the chosen integer type does not overflow. In contrast, many decimal fractions cannot be represented exactly in binary floating point. Repeatedly adding float rupee amounts can accumulate small representation errors.

For example, a bill of ₹162.50 repeated 4,000 times is exactly ₹650,000.00, or 65,000,000 paise. Store the individual bill as 16,250 paise and sum in a sufficiently wide integer. The integer result is exact

**Meter rollover without if–else:**

Expected output on a system where unsigned short is 2 bytes and has maximum 65535:
Sizeof (meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units

The second result is 256 because the 16-bit register wraps modulo 65,536. The difference is 120 − 65,400 = −65,280; interpreted modulo 65,536, this is 256. The cast to unsigned short produces the register-width result. This approach assumes exactly one rollover and the stated register width.

**Coding:**

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   /* Conversion back to meter_t applies modulo 65536 on a
 16-bit register. */
   return (unsigned short)(current - previous);
}

int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
          sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));
   return 0;
}
```

**Alternative: double rupees:**

An alternative is to store amounts as double rupees and round at the billing boundary, for example to two decimal places. Addition remains O(1), just as integer addition does. A double usually occupies 8 bytes, while a long long commonly occupies 8 bytes too. However, binary floating-point cannot represent every decimal amount exactly, and rounding policy must be explicit. Integer paise is preferable when every bill must be exact to the paisa.

When n bills are added, each floating-point operation may introduce a small rounding error. The total error depends on the values and order of operations; it is not safe to assume it will always cancel. Integer paise has zero representation error until overflow occurs.

## TASK 3 – Define the Constants

### Centralised tariff definitions:

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (SLAB2_UNITS - SLAB1_UNITS) *
RATE2_PAISE;
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }
```

```
    /* The first 100 units are free, so no energy charge is added for them. */
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

**Expected output:**

```
    0 units -> Rs 50.00
  100 units -> Rs 50.00
  150 units -> Rs 162.50
  200 units -> Rs 275.00
  350 units -> Rs 950.00
```

## #define, const and enum:

| Feature | Meaning | Best use here |
|---|---|---|
| #define | Preprocessor replaces the token before compilation; it has no C type and no storage of its own. | Simple tariff numbers and compile-time configuration. |
| const variable | A typed object that code is not allowed to modify through that identifier; scope and debugger visibility are clearer. | Typed values, especially local or grouped configuration. |
| enum | Defines named integer constants for a finite set of alternatives. | Customer categories or named states, not fractional tariff rates. |

A macro is easy to update in one place but does not provide normal type checking and may be less convenient to inspect in a debugger. A const object is type-checked and has scope; it may or may not require storage after optimisation. An enum is best for discrete integer choices. Runtime and memory effects of these choices are usually negligible for simple tariff constants; correctness and maintainability are more important.

Changing RATE2_PAISE from 225 to 230 changes every applicable bill consistently because the rate is defined once. After changing it, recompile and rerun all boundary tests.

## TASK 4 – Use Macros Safely

### Safe macro examples:

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 3;
    int b = 7;
    printf("Square: %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(a, b));
    printf("Rupees to paise: %lld\n",
RUPEES_TO_PAISE(50));
    return 0;
}
```

**Expected output:**

```
Square: 16
Maximum: 7
Rupees to paise: 5000
```

**Common macro pitfalls:**

- Without parentheses, #define SQUARE(x) x*x makes SQUARE(a+1) expand to a+1*a+1. With a = 3, this evaluates to 3 + 3 + 1 = 7 rather than 16. Parentheses around the parameter and the whole expression fix precedence.
- MAX(a, b) can evaluate an argument more than once. For example, MAX(i++, j) may increment i during the comparison and again if the first argument is selected. Avoid side effects in macro arguments.
- Macros are expanded by the preprocessor. Run gcc -E file.c -o file.i and inspect file.i to see the expanded source.
- A function is usually better when arguments may have side effects, type checking matters, or the logic is more than a tiny expression.

**Inline function alternative and complexity:**

```
static inline int square_int(int x)
{
    return x * x;
}

static inline int max_int(int a, int b)
{
    return (a > b) ? a : b;
}
```

Both a macro expansion and a function call perform constant-time work for these operations: O(1). A compiler may inline an inline function, but it is not required to do so. A macro's expanded code may be repeated at every use, increasing source and potentially machine-code size when used many times. An inline function has one definition at source level and gives type checking; optimisation may remove call overhead. For this assignment, use macros for simple compile-time constants and prefer functions for reusable calculations.

**4.4 Preprocessor evidence:**

```
gcc -E task4.c -o task4.i
# Open task4.i and locate the expanded SQUARE / MAX expressions
gcc -Wall -Wextra -std=c11 task4.c -o task4
./task4
```

## TASK 5 – Header Files and Conditional Compilation

### 5.1 Example project structure:

```
billing_project/
    main.c
    tariff.c
    billing.h
    README.md
```

### 5.2 Header file: billing.h:

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL

long long compute_bill(int units);
unsigned int consumption(unsigned short previous,
                         unsigned short current);
#endif
```

**5.3 Implementation file: tariff.c:**

```
#include "billing.h"
#include <stdio.h>

unsigned int consumption(unsigned short previous,
                         unsigned short current)
{
    return (unsigned short)(current - previous);
}
long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;
#ifdef DEBUG
    fprintf(stderr, "[DEBUG] units=%d, fixed=%lld paise\n",
            units, paise);
#endif
    if (units > SLAB2_UNITS) {
        paise += (SLAB2_UNITS - SLAB1_UNITS) *
RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) *
RATE3_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) *
RATE2_PAISE;
    }
    return paise;}
```

**5.4 Main file: main.c:**

```
#include <stdio.h>
#include "billing.h"

int main(void)
{
    int units = 150;
    long long paise = compute_bill(units);
    printf("%d units -> Rs %lld.%02lld\n",
           units, paise / 100, paise % 100);
    return 0;
}
```

Compile normally with gcc -Wall -Wextra -std=c11 main.c tariff.c -o billing. Compile with debug output enabled using gcc -Wall -Wextra -std=c11 -DDEBUG main.c tariff.c -o billing_debug. #ifdef DEBUG includes the debugging statements only when DEBUG is defined at compilation.

Include guards prevent a header from being processed repeatedly within one translation unit. #include <file> typically searches the implementation's system include paths; #include "file" typically searches the current/source directory first, then configured include paths.

### 5.5 Compile-time versus run-time debug flag:

With #ifdef DEBUG, a non-debug build has no debug statement in the compiled program and no run-time condition to check. With if (debug), the executable generally retains the debug code and each guarded location performs a condition test. The run-time approach allows debug mode to be switched while the program runs; the preprocessor approach creates separate debug and normal builds. The additional test is O(1) per guarded call.

# TASK 6 – Read and Print Data Correctly:

## 6.1 Format specifiers

| Type / value | printf specifier | scanf specifier | Note |
|---|---|---|---|
| int | %d | %d | Use for ordinary signed integers. |
| unsigned int | %u | %u | Non-negative integer. |
| long | %ld | %ld | Specifier must match the argument type. |
| long long | %lld | %lld | Suitable for wide totals. |
| float | %f | %f | In printf, float is promoted to double; %f is used. |
| double | %f | %lf | For scanf, %lf reads a double. |
| char | %c | %c | Whitespace may need to be skipped with a leading space: " %c". |
| string | %s | %s | Use a width limit or prefer fgets for lines. |

**6.2 Validate scanf return values:**

```
#include <stdio.h>

int main(void)
{
    int previous, current;
    printf("Enter previous and current readings: ");

    if (scanf("%d %d", &previous, &current) != 2) {
        fprintf(stderr, "Invalid input: enter two integers.\n");
        return 1;
    }
    if (previous < 0 || previous > 65535 ||
        current < 0 || current > 65535) {
        fprintf(stderr, "Readings must be between 0 and
65535.\n");
        return 1;
    }

    printf("Readings accepted: %d and %d\n", previous,
current);
    return 0;
}
```

.

scanf returns the number of successfully assigned input items. If the user enters letters instead of numbers, the return value will be less than the expected count and the program should reject the input. Using a wrong format specifier, such as %d for a long argument in printf, causes undefined behaviour; the output cannot be relied on

**6.3 Print rupees and paise:**

```
long long paise = 16250;
printf("Rs %lld.%02lld\n", paise / 100, paise % 100);
/* Output: Rs 162.50 */
```

**6.4 fgets plus conversion alternative:**

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

int parse_reading(const char *line, long *value)
{
```

## TASK 7 – Control Type Conversion and Precision

### 7.1 Where conversions happen

```
    char *end;
    errno = 0;
    long v = strtol(line, &end, 10);
    if (line == end || errno == ERANGE || v < 0 || v > 65535)
        return 0;
    while (*end == ' ' || *end == '\\t' || *end == '\\r' || *end == '\\n')
        end++;
    if (*end != '\\0')
        return 0;
    *value = v;
    return 1;
}
```

fgets reads a whole line into a fixed-size buffer, after which strtol can validate conversion and range. Processing a line of length L is O(L), and buffer space is O(L) up to the chosen maximum line length. scanf can be concise, but line-based input is often easier to recover from because the whole invalid line can be discarded before the next attempt.

## TASK 7 – Control Type Conversion and Precision:

### 7.1 Where conversions happen:

- When short or unsigned short operands are used in arithmetic, integer promotions usually convert them to int if int can represent every value of the original type.
- The usual arithmetic conversions decide the common type when different numeric types are combined.
- Integer division discards the fractional part when both operands are integers.
- An explicit cast can force conversion, but casting a value outside the destination type's range may lose information; converting an out-of-range signed integer to a smaller signed type is implementation-defined or may raise an implementation-defined signal.
- When converting fractional charges to paise, choose a documented rule, such as rounding to the nearest paise, and apply it once at the defined billing boundary.

## 7.2 Three expressions that can surprise:

| Expression | Result | Explanation |
|---|---|---|
| 5 / 2 | 2 | Both operands are integers, so the fractional part is discarded. |
| (double)(5 / 2) | 2.0 | The division happens first as integer division; casting afterwards cannot recover the .5. |
| (double)5 / 2 | 2.5 | One operand is converted before division, so floating-point division is used. |
| 0.1 + 0.2 == 0.3 | Often false in binary floating point | The decimal fractions are not represented exactly in binary; compare with a tolerance when floating point is appropriate. |

### 7.3 Example of a narrow cast:

```
long long total_paise = 76000000000LL;
int narrowed = (int)total_paise; /* value is outside 32-bit int range */
printf("%d\n", narrowed);        /* do not rely on a particular result */
```

The example demonstrates why district totals must not be narrowed to int. Use a wide type for the total and match format specifiers to the exact type.

### 7.4 Float totals versus integer paise:

If all calculations were performed in float rupees and rounded only at the end, each charge would be subject to binary representation error and the running total would have limited precision. A float typically has about 6–7 decimal digits of precision. At large aggregate values, its spacing between representable numbers becomes much larger than one paisa, so a total can lose small increments.

Integer paise preserves exact cents as long as arithmetic does not overflow. For both approaches, adding n bills takes O(n) time overall and O(1) extra space for a running total. A float value commonly occupies 4 bytes; a long long commonly occupies 8 bytes. The integer approach is recommended for billing.

# TASK 8 – Trace the Compilation Process:

## 8.1 GCC stages and outputs:

```
# Preprocess: expand macros and include header contents
gcc -E main.c -o main.i

# Compile to assembly
gcc -S main.c -o main.s

# Compile to object file
gcc -c main.c -o main.o

# Link source files into an executable
gcc main.c tariff.c -o billing

# Enable useful warnings
gcc -Wall -Wextra -Wpedantic -std=c11 main.c tariff.c -o billing
```

| Stage | Command / output | What happens |
|---|---|---|
| Preprocessing | gcc -E; .i file | Includes headers, expands macros and processes conditional compilation. |
| Compilation to assembly | gcc -S; .s file | Checks and translates C into assembly language. |
| Assembly | gcc -c; .o object file | Creates machine-code object files; external references may remain unresolved. |
| Linking | gcc main.c tariff.c -o billing | Combines object code and libraries into an executable. |

## 8.2 Error categories:

- Compile-time error: a syntax error such as a missing semicolon or an undeclared identifier.
- Link-time error: calling a function declared but not defined in any linked source/object file, leading to an undefined reference or unresolved external.
- Run-time error: invalid input, arithmetic overflow in an unchecked calculation, or a program crash while executing. A logical error such as applying the wrong slab may run without crashing but still produce an incorrect bill.

Warnings from -Wall -Wextra -Wpedantic can reveal unused variables, suspicious conversions, format-specifier mismatches, or non-standard code. Resolve warnings rather than suppressing them without understanding the cause.

**8.3 One file versus multiple files:**

In a single-file program, changing any part normally requires recompiling the entire translation unit. In a multi-file project, only changed source files need to be recompiled, then the object files are linked again. The total object-file space is approximately the sum of the individual object files, plus build metadata. Splitting the program adds organisation and build steps but helps larger projects.

```
gcc -Wall -Wextra -std=c11 -c main.c -o main.o
gcc -Wall -Wextra -std=c11 -c tariff.c -o tariff.o
gcc main.o tariff.o -o billing
```

# TASK 9 – Implement and Test the Billing Program

## 9.1 Complete program:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL

typedef unsigned short meter_t;

static unsigned int consumption(meter_t previous, meter_t
current)
{
    return (unsigned short)(current - previous);
}
```

```
static long long compute_bill(unsigned int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (SLAB2_UNITS - SLAB1_UNITS) *
RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) *
RATE3_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) *
RATE2_PAISE;
    }
    return paise;
}

int main(void)
{
    int previous, current;
    printf("Enter previous and current meter readings (0-
65535): ");

    if (scanf("%d %d", &previous, &current) != 2) {
        fprintf(stderr, "Error: enter two integer readings.\n");
        return 1;
```

```
}
    if (previous < 0 || previous > USHRT_MAX ||
        current < 0 || current > USHRT_MAX) {
        fprintf(stderr, "Error: reading out of range.\n");
        return 1;
    }

    unsigned int units = consumption((meter_t)previous,
(meter_t)current);
    long long bill = compute_bill(units);

    printf("Units consumed: %u\n", units);
    printf("Energy charge and fixed charge included.\n");
    printf("Total bill: Rs %lld.%02lld\n", bill / 100, bill %
100);
    return 0;
}
```

This program treats the 16-bit register as a modulo counter. Since two readings alone cannot distinguish rollover from a meter reset or data-entry error, a production system should validate the meter's status and rollover assumptions. The brief explicitly asks for rollover handling, so the code follows that rule.

## 9.2 Test plan and expected results

| Test | Input / condition | Expected result | Status to record |
|---|---|---|---|
| Slab 1 lower boundary | 0 units | ₹50.00 total | Run and fill actual |
| Exactly 100 units | 100 units | ₹50.00 total | Run and fill actual |
| Within slab 2 | 150 units | ₹162.50 total | Run and fill actual |
| Exactly 200 units | 200 units | ₹275.00 total | Run and fill actual |
| Above 200 units | 350 units | ₹950.00 total | Run and fill actual |
| Meter rollover | previous=65400, current=120 | 256 units consumed; bill ₹527.00 | Run and fill actual |
| No consumption | previous=1200, current=1200 | 0 units; bill ₹50.00 | Run and fill actual |
| Invalid letters | previous=a, current=120 | Reject input; no bill | Run and fill actual |
| Negative reading | previous=-1, current=120 | Reject input; no bill | Run and fill actual |
| Out-of-range reading | previous=100, current=70000 | Reject input; no bill | Run and fill actual |

Rollover bill calculation: 256 units means 100 units free, 100 units × ₹2.25 = ₹225, and 56 units × ₹4.50 = ₹252. Energy charge = ₹477; plus ₹50 fixed charge gives ₹527.00. This corrected calculation should be used in the test table.

## 9.3 Five-bill hand calculation

| Household | Units | Energy charge | Fixed charge | Total |
|---|---|---|---|---|
| A | 0 | ₹0.00 | ₹50.00 | ₹50.00 |
| B | 100 | ₹0.00 | ₹50.00 | ₹50.00 |
| C | 150 | ₹112.50 | ₹50.00 | ₹162.50 |
| D | 200 | ₹225.00 | ₹50.00 | ₹275.00 |
| E | 350 | ₹900.00 | ₹50.00 | ₹950.00 |
| Total | — | ₹1,237.50 | ₹250.00 | ₹1,487.50 |

Array-based slab alternative: store slab limits and rates in arrays, then loop over s slabs. Time per bill becomes O(s), and extra storage is O(s) for the tariff table. This is easier to extend when many slabs exist. A fixed sequence of if statements is simpler and effectively constant time when there are only three slabs. A production design should also reject negative unit counts and validate the meter-reading scenario.

## TASK 10 – Compare Expectations with Execution

### 10.1 Aggregate totals at village, taluk and district scale:

For this comparison, let us assume that each household receives an electricity bill of ₹950. This is only an example used to check the program's performance.

| Scale | Number of bills | Total in paise |
|---|---|---|
| Village | 4,000 | 380,000,000 |
| Taluk | 60,000 | 5,700,000,000 |
| District | 800,000 | 76,000,000,000 |

A normal 32-bit signed int cannot store the taluk and district totals in paise because the values are too large. A 64-bit long long can store these totals safely. A float becomes less accurate as the total increases, while a double provides better precision but cannot represent every decimal value exactly.

Before submitting, run the program on your computer and record the actual results.

| Measurement | Actual result |
| --- | --- |
| Compiler and version | _____ |
| sizeof(int), INT_MAX | _____ |
| sizeof(long), LONG_MAX | _____ |
| sizeof(long long), LLONG_MAX | _____ |
| sizeof(float), FLT_DIG | _____ |
| sizeof(double), DBL_DIG | _____ |

**TASK 11 – Recommend and Defend the Final Design:**

For this billing program, I recommend using unsigned short for the 16-bit meter reading, unsigned int for units consumed, and long long for bill amounts and the total collection. These data types are suitable for the meter readings and large bill totals in this assignment.

The tariff values should be defined in one place so that any changes can be made easily. Separate functions should be used for calculating consumption and bills, and all user inputs should be validated to prevent incorrect results. Storing money as integer paise avoids decimal rounding errors. A 64-bit long long can store the example district total of 76,000,000,000 paise.

The main limitation is that the basic design assumes at most one meter rollover between readings and fixed tariff rates. Additional rules, such as taxes or discounts, would require further changes.

If tariff values are stored in a configuration file, the program can read them when it starts. Reading t tariff values takes O(t) time and O(t) space. This allows tariff changes without recompiling the program, but the file must be checked for invalid values.

**TASK 12 – Day-4 Smart-Meter Design Challenge**

**A. Impact Analysis**

In the updated system, smart meters send readings in watt-hours every 15 minutes. Electricity used between 6 PM and 10 PM costs 20% more than electricity used during other hours. The tariff rates must also be changeable without recompiling the program.

Each house produces:

35,040 readings per year.

| Scale | Annual readings |
|---|---|
| 4,000 houses | 140,160,000 |
| 60,000 houses | 2,102,400,000 |
| 800,000 houses | 28,032,000,000 |

A 32-bit signed int is not large enough to store the district's annual reading count. Therefore, a 64-bit counter and wider energy totals should be used.

Hard-coded tariff macros are no longer suitable because the rates must be changed while keeping the same executable.

**B. Evaluation of the Original Design:**

The original design uses integer paise to avoid rounding errors, which is still a good choice. However, 16-bit meter readings are not suitable for storing large cumulative energy totals.
The original design also uses fixed tariff values. These need to be replaced with values read from a configuration file so that tariff changes can be made without recompiling the program.

**C. Revised Design:**

The revised program can read the base tariff in paise per kWh from a configuration file. It then applies a multiplier of 120 during peak hours and 100 during other hours.

C program:

```
#include <stdio.h>
#include <stdint.h>
int64_t calculate_charge(uint64_t wh, int peak,
int64_t rate)
{
int64_t multiplier = peak ? 120 : 100;
return (int64_t)((wh * (uint64_t)rate *
(uint64_t)multiplier + 50000ULL)
/ 100000ULL);
}
```

```
int main(void)
{
printf("Charge: %lld paise\n",
(long long)calculate_charge(1500, 1, 225));
return 0;
}
```

**Expected output:**

Charge: 405 paise

The charge is ₹4.05 for 1,500 Wh at a rate of 225 paise per kWh during peak hours. The example rounds the charge to the nearest paisa. In a real system, the multiplication should also be checked to prevent overflow.

**D. Evidence-Based Defence:**

The revised design continues to store money as integer paise to keep the calculations accurate. It uses 64-bit counters to handle the large number of smart-meter readings. Tariff rates can be updated through a configuration file, and the program applies the higher rate during peak hours.

Processing N readings takes O(N) time. Saving readings as they arrive also avoids keeping all the readings for an entire year in memory.

**Conclusion:**

The billing program uses suitable data types, stores money in paise, keeps tariff values in one place, and checks user input. A wide integer variable helps prevent overflow when calculating the total collection.

For the smart-meter system, the design needs to support larger reading counts, tariff rates that can be changed without recompiling, peak-hour pricing, and clear rules for rounding and preventing overflow.